Last Time:

- Attitude Regulation
- Eigen-Axis Slew

Today:

- Time-Varying LQR

TVLQR:

- Last time we covered how to generate open-loop trajectories.
- We still need a feedback controller to track online
- The combination of offline trajectory planning with online LQR tracking is very common + effective.

* Problem Setup:

$$\min_{x_{1:N},\, u_{1:N-1}} \sum_{n=1}^{N-1} \frac{1}{2} x_n^T Q x_n + \frac{1}{2} u_n^T R u_n + \frac{1}{2} x_N^T Q_N x_N$$

$$\text{s.t.} \quad x_{n+1} = A_n x_n + B_n u_n$$

* Dynamic Programming Solution:

- Define "cost-to-go" function $V_n(x)$
- $V_n(x)$ gives the minimum/optimal to drive the system from state x at time t_n to the goal at t_N

- Start at t_N and work backwards:

$$V_N(x) = \frac{1}{2}x^T Q_N x = x^T S_N x$$

- Back one time step:

$$V_{N-1}(x) = \min_u \left[\frac{1}{2}x^T Q x + \frac{1}{2}u^T R u + \frac{1}{2}(A_{N-1}x + B_{N-1}u)^T S_N (A_{N-1}x + B_{N-1}u) \right]$$

$$\frac{\partial}{\partial u}(\,) = u^T R + u^T B_{N-1}^T S_N B_{N-1} + x^T A_{N-1}^T S_N B_{N-1} = 0$$

$$\Rightarrow \boxed{u = -\underbrace{(R + B_{N-1}^T S B_{N-1})^{-1} B_{N-1}^T S_N A_{N-1}}_{K_{N-1}} x}$$

- Now plug $u = -Kx$ back in to get $V_{N-1}(x)$:

$$V_{N-1}(x) = x^T Q x + x^T K^T R K x + x^T (A - BK)^T S_N (A - BK) x$$

$$\Rightarrow \boxed{S_{N-1} = Q + K_{N-1}^T R K_{N-1} + (A_{N-1} - B_{N-1}K_{N-1})^T S_N (A_{N-1} - B_{N-1}K_{N-1})}$$

- We keep doing this recursively until we get to $k = 1$
- Called "Riccati Recursion" - Special case of Bellman Equation

---

TVLQR Algorithm Summary:

1) Initialize $S_N = Q_N$
2) Compute $K_n = (R + B_n^T S_{n+1} B_n)^{-1} B_n^T S_{n+1} A_n$
3) Compute $S_n = Q + K_n^T R K_n + (A_n - B_n K_n)^T S_{n+1} (A_n - B_n K_n)$
4) While $k > 1$ go to 2

- Define Q and R matrices to trade off tracking error vs. control effort. Often we use diagonal weights.

- A good starting guess is "Bryson's Rule":

$$Q = \text{diag}\left(\left(1/\text{max deviation}\right)^2 \dots\right)$$

$$R = \text{diag}\left(\left(1/\text{max control}\right)^2 \dots\right)$$

* This attempts to normalize cost terms to ~ 1

- Perform a 1st order Taylor expansion along the nominal trajectory:

$$x_{n+1} + \Delta x_{n+1} \approx f(x_n, u_n) + \underbrace{\frac{\partial f}{\partial x}\Big|_{x_n,u_n}}_{A_n} \Delta x_n + \underbrace{\frac{\partial f}{\partial u}\Big|_{x_n,u_n}}_{B_n} \Delta u_n$$

$$\Rightarrow \Delta x_{n+1} = A_n \Delta x_n + B_n \Delta u_n$$

- Compute TVLQR gain matrices for this linear system ("error dynamics")

- Outline control law:

$$u_n = \underbrace{\bar{u}_n}_{\text{nominal (feed-forward)}} - \underbrace{K_n \Delta x_n}_{\text{feedback "correction"}}$$

(u_n: actual command)

- For a system where $x \in \mathbb{R}^n$, $\Delta x_n = \underbrace{x_n}_{\text{actual}} - \underbrace{\bar{x}_n}_{\text{plan}}$ (error)

- When x includes a quaternion, we have to do some extra work

LQR with Quaternions:

- We'll apply our quaternion differentiation tricks to LQR
- Given a reference $\bar{x}_n$, $\bar{u}_n$ for discrete-time systems:

$$\bar{x}_{n+1} + \delta x_{n+1} = f(\bar{x}_n + \delta x_n, \bar{u}_n + \delta u_n)$$

$$\approx f(\bar{x}_n, \bar{u}_n) + A_n \delta x_n + B_n \delta u_n$$

- For the quaternion part of the state, we apply the attitude Jacobian to convert $\delta q \to \phi \in \mathbb{R}^3$

$$x = \begin{bmatrix} q \\ \omega \\ \vdots \end{bmatrix}$$

$$\underbrace{\begin{bmatrix} \phi_{n+1} \\ \omega_{n+1} \\ \vdots \end{bmatrix}}_{\delta x_{n+1}} = \underbrace{\begin{bmatrix} G(\bar{q}_{n+1}) & 0 & 0 \\ 0 & I & 0 \\ 0 & 0 & I \end{bmatrix}^T}_{E(\bar{x}_{n+1})} A_n \underbrace{\begin{bmatrix} G(\bar{q}_n) & 0 & 0 \\ 0 & I & 0 \\ 0 & 0 & I \end{bmatrix}}_{E(\bar{x}_n)} \underbrace{\begin{bmatrix} \phi_n \\ \omega_n \\ \vdots \end{bmatrix}}_{\delta x_n}$$

$$\underbrace{E(\bar{x}_{n+1})^T A_n E(\bar{x}_n)}_{\tilde{A}_n} + \underbrace{E(\bar{x}_{n+1})^T B_n}_{\tilde{B}_n} \delta u_n$$

- Once we have these "reduced" Jacobians $\tilde{A}_n$, $\tilde{B}_n$, we compute the TVLQR as usual.

- When we run the controller, we calculate δx before multiplying by K:

given x_n, $\delta x_n = \begin{bmatrix} \phi(L(\bar{q}_n)^T q_n) \\ \omega_n - \bar{\omega}_n \\ \vdots \end{bmatrix}$

$$u_n = \bar{u}_n - K_n \delta x_n$$

* Computing error/delta rotations:

- Rotation from body frame B to the reference/desired body frame D:

$$\Rightarrow {}^N_B Q = {}^N_D Q \, {}^D_B Q \qquad \Rightarrow ({}^N_D Q)^{-1} \, {}^N_B Q = {}^D_B Q$$

$$Q = \bar{Q} \, \delta Q \qquad \Rightarrow \delta Q = \bar{Q}^T Q$$

- For quaternions:

$$\delta q = \bar{q}^+ * q = L(\bar{q})^T q$$

TVLQR for Attitude Tracking:

- Continuous-Time Dynamics:

$$\dot{x} = \begin{bmatrix} \dot{q} \\ \dot{\omega} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} \frac{1}{2} G(q)\,\omega \\ -J^{-1}\left[\omega \times (J\omega + B_\omega r) + B_\omega u\right] \end{bmatrix}$$

wheel Jacobian

u: wheel torques

$\dot{r}$: individual wheel momenta

$(\rho = B_\omega r)$

- Linearization:

$$\delta\dot{x} = \begin{bmatrix} \dot{\phi} \\ \delta\dot{\omega} \\ \delta\dot{r} \end{bmatrix} = \underbrace{\begin{bmatrix} -\hat{\omega} & I & 0 \\ 0 & -J^{-1}\left[\hat{\omega}J - (J\omega + B_r r)\hat{}\right] & -J^{-1}\hat{\omega}B_r \\ 0 & 0 & 0 \end{bmatrix}}_{\tilde{A}(t)} \begin{bmatrix} \phi \\ \delta\omega \\ \delta r \end{bmatrix}$$

$$+ \underbrace{\begin{bmatrix} 0 \\ -J^{-1}B_\omega \\ I \end{bmatrix}}_{\tilde{B}(t)} u$$

- Convert to discrete time with zero-order hold:

$$\delta x_{n+1} = \tilde{A}_n \delta x_n + \tilde{B}_n \delta u_n$$

$$\begin{bmatrix} \tilde{A}_n & \tilde{B}_n \\ 0 & I \end{bmatrix} = e^{\begin{bmatrix} \tilde{A}(t_n) & \tilde{B}(t_n) \\ 0 & 0 \end{bmatrix} \Delta t}$$

- Often, 1st-order Taylor expansion is used if Δt is small:

$$\tilde{A}_n \simeq I + \tilde{A}(t_n)\Delta t \quad , \quad \tilde{B}_n \simeq \tilde{B}(t_n)\Delta t$$

- Calculate K_n as usual using Riccati (offline)

* Online:

- Calculate error vs. plan: $\delta x_n = \begin{bmatrix} \phi\left(L^T(\bar{q}_n) q_n\right) \\ \omega_n - \bar{\omega}_n \\ r_n - \bar{r}_n \end{bmatrix}$

- Apply feedforward + feedback control:

$$u_n = \bar{u}_n - K_n \delta x_n$$